

Indexing and Retrieval systems for Audio

Hoa Dinh Nguyen, PhD.

hoand@ptit.edu.vn

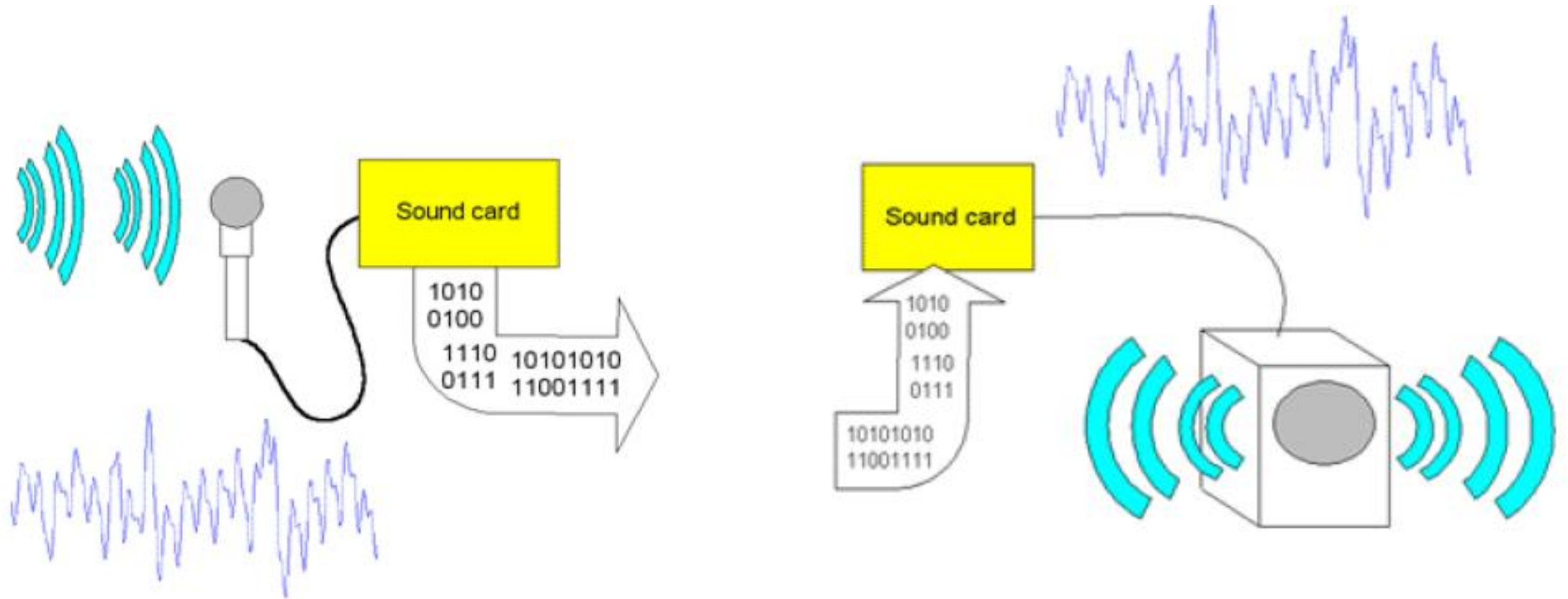
Lecture Overview: Audio Indexing and Retrieval

- ▶ Motivation and approaches: speech, music, noise; CBAR framework
- ▶ Audio features: time-domain and frequency-domain descriptors
- ▶ Classification and segmentation: step-wise filters, attribute vectors
- ▶ Speech retrieval: ASR + text IR (link to Lecture 8)
- ▶ Music retrieval: structured (MIDI) vs sampling-based (MP3) + query-by-humming
- ▶ Goal: index audio by content and enable similarity-based search in a multimedia DB



Audio data

- ▶ Audio is stored in a sequence of samples, usually in compressed form



Audio indexing and retrieval

- ▶ Some situations to solve:
 - ▶ If we have an audio file of an actor's performance, how do we know when the actor is singing, when the actor is talking to the audience?
 - ▶ If we have a recording of a scientific conference, how do we know exactly when people are talking about a particular XYZ issue?
 - ▶ If we have a lot of music files, how can we find the music files we are looking for if we don't remember the name of the song but only the melody?
 - ▶ If we need to find horror scenes in a movie, how do we quickly find them based on the sound in the movie?

Audio indexing and retrieval

▶ Basic approaches

- ▶ Audio is classified into one of 3 main types: speech, music, noise
- ▶ Each type of audio is processed, stored, and queried in different ways
 - ▶ For example, speech is analyzed/identified into words, and these words are stored.
- ▶ Different types of sounds have different meanings according to specific applications
- ▶ Audio classification usually makes data querying more efficient.
- ▶ Sounds are stored and queried based on similarity comparisons on their attributes.

Content-Based Audio Retrieval (CBAR) Framework

- ▶ Offline: audio file → decode → frame → extract features → build index → store in MMDB
- ▶ Online: query (audio clip, humming, keyword) → extract features → similarity search → ranked results
- ▶ Index types: metadata catalog (ID3 tags), feature vector index, inverted text index (ASR transcript)
- ▶ Similarity measures: Euclidean, cosine, DTW (Dynamic Time Warping) for temporal alignment
- ▶ Semantic gap: low-level features (MFCC) vs user query ('sad song', 'horror scene') — classification helps
- ▶ Three audio classes drive processing path: speech → ASR; music → pitch/timbre; noise → event detection



Audio features

- ▶ Time domain features
 - ▶ Average energy
 - ▶ Zero crossing rate
 - ▶ Silence ratio
- ▶ Frequency domain features
 - ▶ Sound spectrum
 - ▶ Bandwidth
 - ▶ Sound Energy Distribution
 - ▶ Harmonicity
 - ▶ Pitch
- ▶ Spectrogram

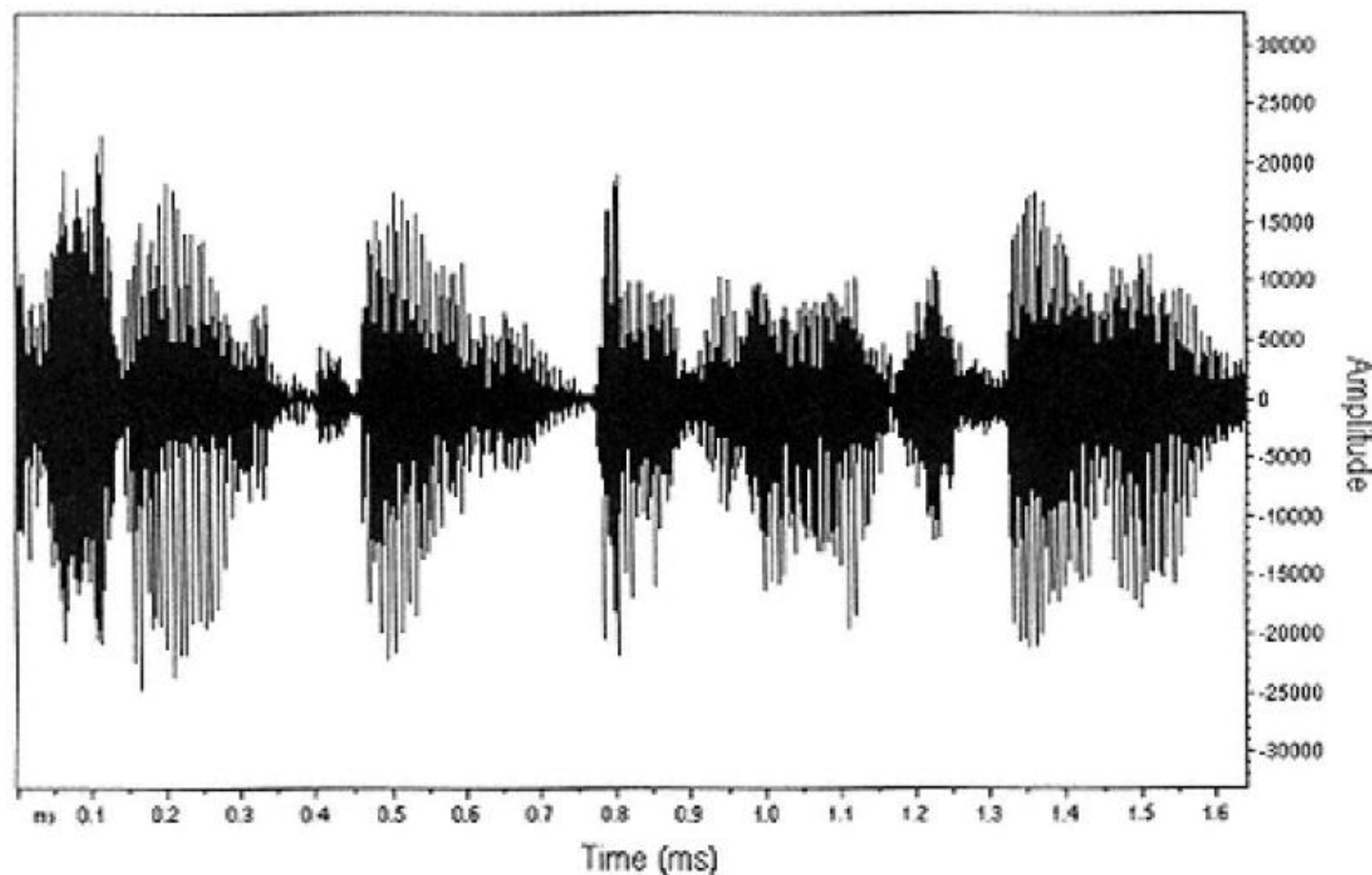
Audio Feature Extraction Pipeline

- ▶ Preprocessing: decode compressed audio (MP3/AAC) → mono PCM at standard rate (16 kHz speech, 44.1 kHz music)
- ▶ Framing: split into overlapping windows (e.g., 25 ms frame, 10 ms hop) for short-time analysis
- ▶ Time-domain features: computed per frame or globally (energy, ZCR, silence ratio)
- ▶ Frequency-domain: apply FFT/STFT per frame → spectrum, centroid, bandwidth, harmonicity, pitch
- ▶ Aggregation: mean/variance of frame features over segment → segment-level feature vector
- ▶ Storage: feature vector + time offsets stored as index entry linked to audio OID in MMDB



Time domain features

- Variation of sound intensity over time



Time domain features

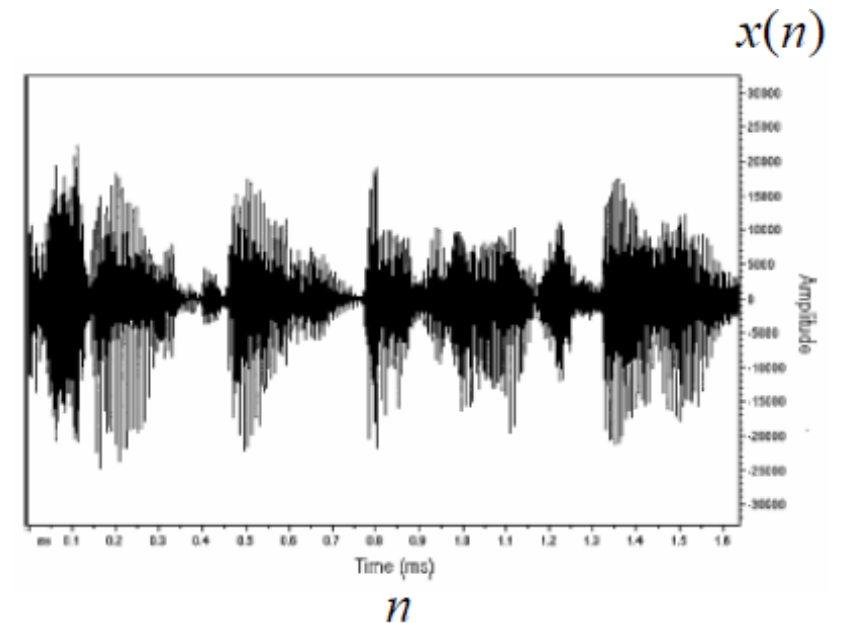
- ▶ Average energy: expressing the loudness of the sound

$$E = \frac{\sum_{n=0}^{N-1} x(n)^2}{N}$$

- ▶ Zero crossing rate: represents the mean frequency of signal

$$ZC = \frac{\sum_{n=1}^N |sgn[x(n)] - sgn[x(n-1)]|}{2N}$$

$$sgn(a) = \begin{cases} 1 & a > 0 \\ 0 & a = 0 \\ -1 & a < 0 \end{cases}$$



Time domain features

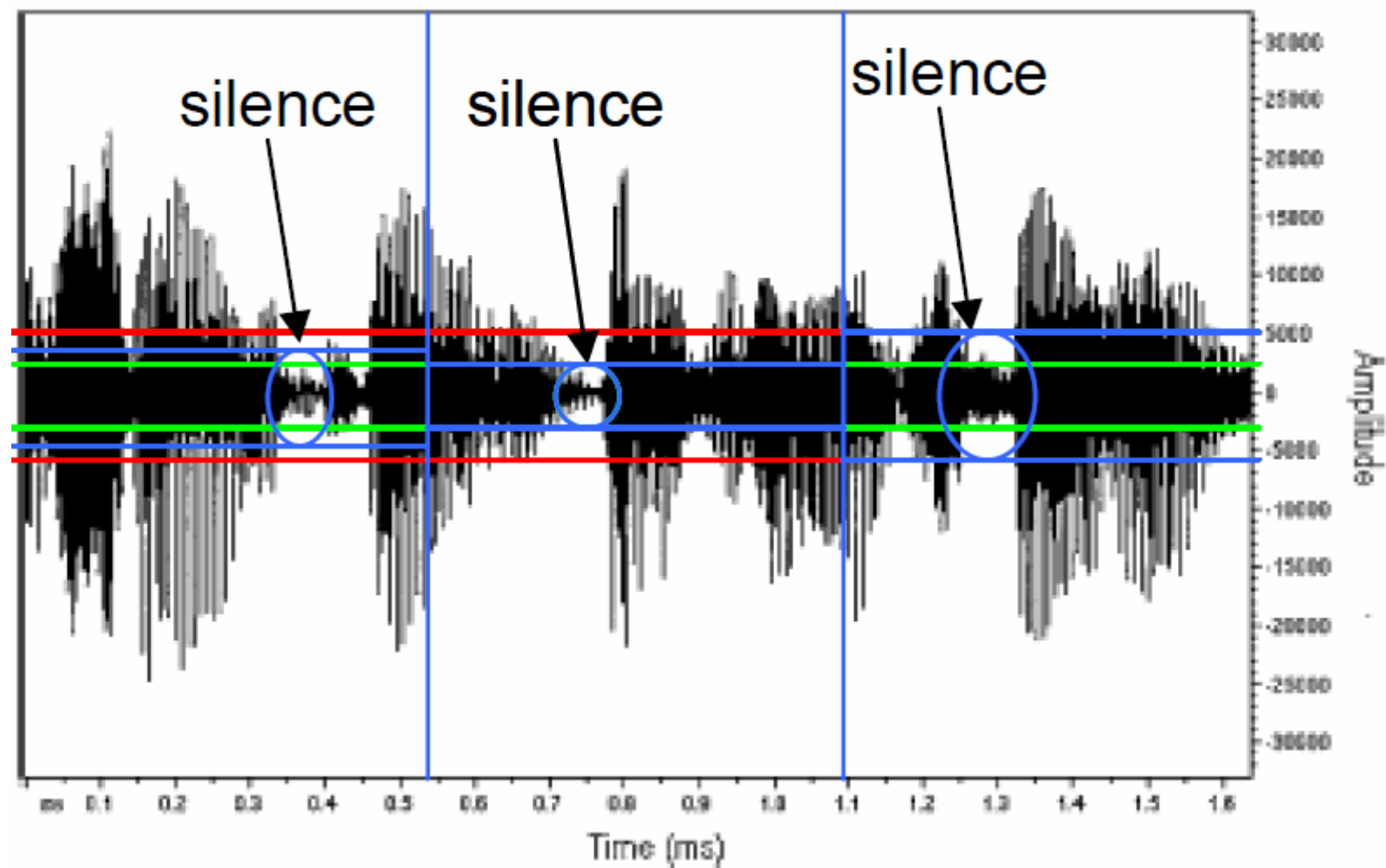
▶ Silence ratio

- ▶ Silence is the period of time during which the absolute value of the sound wave amplitude is less than a given threshold
- ▶ The silence ratio is the ratio of the total silence time divided by the entire length of the audio file
- ▶ There are 2 threshold values to be determined: the amplitude threshold and the time threshold.
- ▶ Ways to determine the amplitude threshold:
 - ▶ Use a given fixed threshold value
 - ▶ Use relative thresholds, based on an available value
 - ▶ Use adaptive threshold values.

Time-Domain Feature Formulas

- ▶ Average energy (per frame): $E = (1/N) \sum x[n]^2$ — loudness indicator
- ▶ Zero-crossing rate (ZCR): $ZCR = (1/2N) \sum |sign(x[n]) - sign(x[n-1])|$ — correlates with frequency
 - ▶ High ZCR → noisy/unvoiced/high-frequency content; low ZCR → voiced speech or tonal music
- ▶ Silence ratio: total silence duration / total duration
 - ▶ Silence if $|x[n]| < A_{thresh}$ for at least T_{min} consecutive samples
- ▶ Example: 60 s clip, 8 s silence → silence ratio = $8/60 \approx 0.133$

Time domain features



Frequency domain features

- ▶ Sound spectrum: using Fourier transforms or other transform methods
 - ▶ Discrete Fourier transformations (DFT)

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-\frac{j2\pi nk}{N}}$$

$X(k)$ is periodic with frequency $f_k = f_s \frac{\omega_k}{N} = f_s \frac{k}{N}$

- ▶ Inverse discrete Fourier transformation (IDFT)

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{\frac{j2\pi nk}{N}}$$

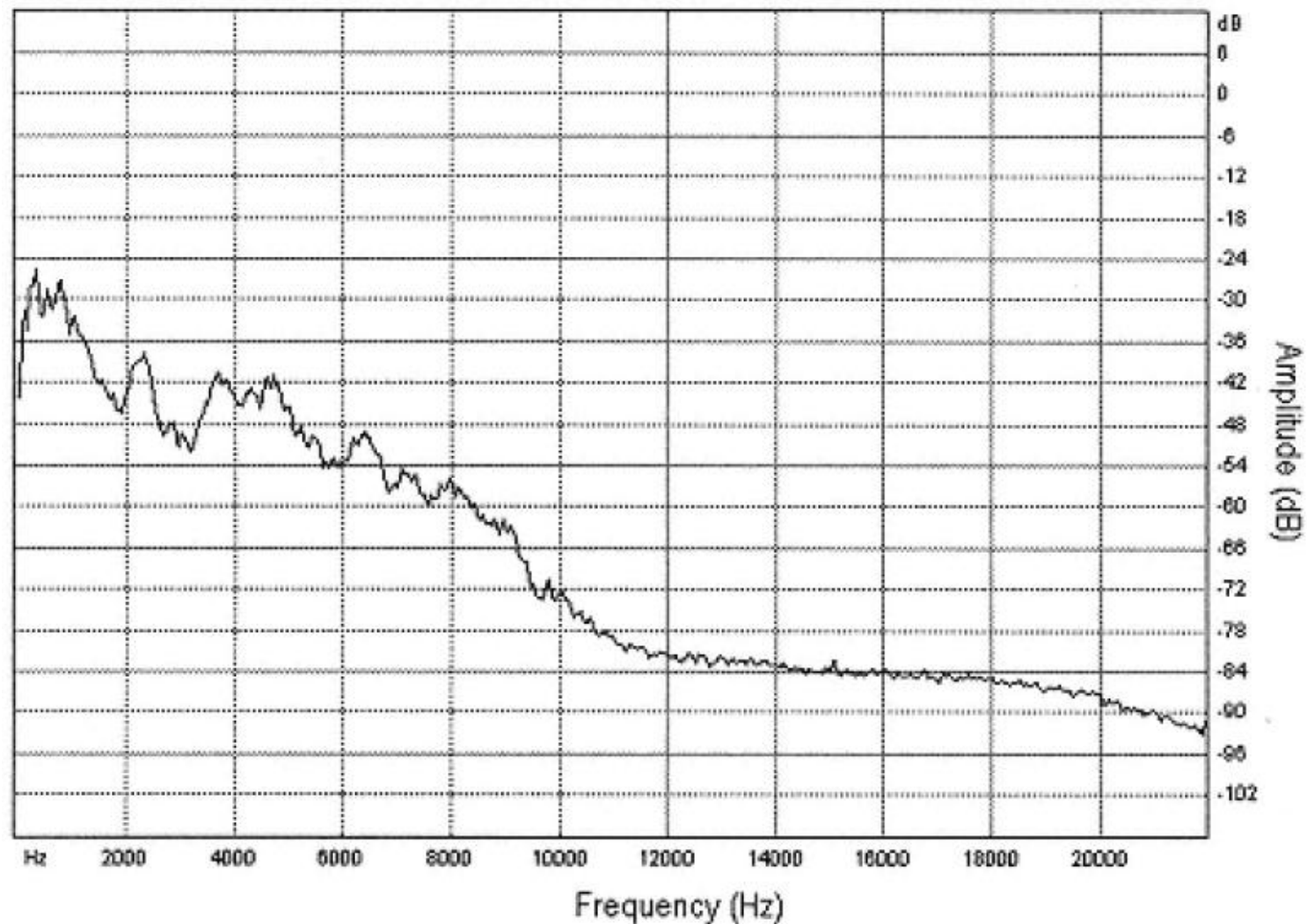
If N is large, the signal is divided into small segments (frames), DFT is performed on these segments – called short-time Fourier transformation.

DFT, STFT, and Frequency Resolution

- ▶ DFT: maps N time samples to N frequency bins
- ▶ Frequency bin spacing: $\Delta f = \frac{f_s}{N}$ (e.g., $f_s=8000$ Hz, $N=512 \rightarrow \Delta f \approx 15.6$ Hz)
- ▶ STFT: apply DFT to each overlapping frame $\rightarrow$ time-frequency representation
- ▶ Trade-off: longer frame $\rightarrow$ finer frequency resolution, poorer time resolution (and vice versa)
- ▶ Spectrogram: $|\text{STFT}(t,f)|$ displayed as 2D image (time $\times$ frequency, color = magnitude)
- ▶ Window function (Hamming, Hann) reduces spectral leakage at frame boundaries



Spectral examples of an audio signal



Frequency domain features

▶ Bandwidth

- ▶ Is the frequency range of the sound
- ▶ Calculated by the difference between the highest and lowest frequencies of the positive spectrum components.
- ▶ "positive" is defined at about 3dB above the mute level.

▶ Energy Distribution

- ▶ Is the distribution of signal energy along the frequency components
- ▶ Usually, music signals have higher frequencies than speech signals. The frequency of the speech signal is usually no more than 7kHz.
- ▶ Based on the distribution of sound energy, we can determine the centroid of the sound, which is the midpoint of the sound energy. The centroid of the sound is also known as the brightness.

Spectral Centroid, MFCC, and Harmonicity

- ▶ Spectral centroid (brightness): $SC = \frac{\sum f_k \times |X[k]|}{\sum |X[k]|}$, weighted mean frequency
- ▶ Harmonicity: detect peaks at $f, 2f, 3f, \dots$ — high harmonic ratio → musical instrument or vowel
- ▶ MFCC (Mel-Frequency Cepstral Coefficients): standard timbre descriptor for CBAR
 - ▶ Steps: STFT → Mel filter bank → log → DCT → 13 MFCC coefficients per frame
- ▶ MFCCs capture timbre; used in speech recognition, music genre classification, audio fingerprinting
- ▶ Pitch (f_0): fundamental frequency from autocorrelation, cepstrum, or YIN algorithm



Các thuộc tính miền tần số

▶ Harmonicity

- ▶ In toned sounds, the spectral components mostly consist of the coefficients of the lowest (usually loudest) frequency of the sound.
- ▶ The lowest frequency of the sound is called the fundamental frequency
- ▶ Music often has greater harmonics than other sounds.
- ▶ For example, the sound of a flute usually has a frequency of sound spectrum peaks of 400Hz, 800Hz, 1200Hz, 1600Hz, etc. corresponding to f , $2f$, $3f$, $4f$,... where $f=400\text{Hz}$ is the basic frequency of the flute.

Frequency domain features

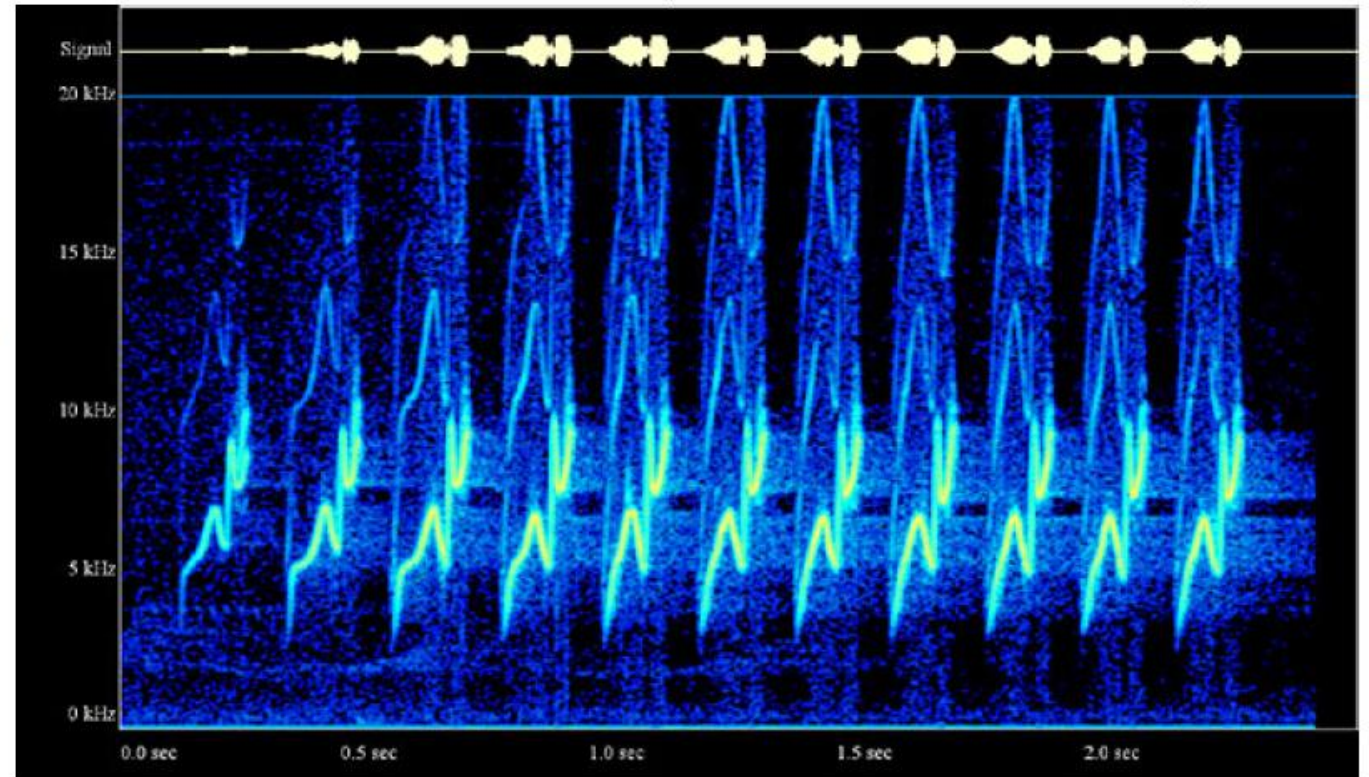
▶ Pitch

- ▶ It is a unique characteristic of sound quality, depending on the frequency of the sound source
- ▶ Only cyclic sounds, or sounds produced by musical instruments, can be high-pitched
- ▶ Fundamental frequencies are often used to estimate the pitch of a sound

Spectrogram

- ▶ It is a form of simultaneous expression of components in the frequency and time domains of sound

frequency



time

Intensity: Power of a frequency component at a particular time interval

Reading a Spectrogram

- ▶ Horizontal axis: time; vertical axis: frequency; color/intensity: energy magnitude
- ▶ Horizontal bands → harmonic tones (music, vowels); vertical striations → speech formants
- ▶ Silence → dark/low-energy regions; transients (drums) → vertical broadband energy bursts
- ▶ Horror scene audio: sudden high-energy dissonance, low-frequency rumble, screaming (2–4 kHz energy)
- ▶ Segmentation cue: spectrogram boundary changes indicate audio type transitions
- ▶ Used in audio editing, bird song analysis, and query-by-example retrieval



Comparison between speech and music

Features	Speech	Music
Bandwidth	0–7 kHz	0–20 kHz
Spectral centroid	low	high
Silence ratio	high	low
Zero-crossing rate	more variable	less variable
Regular beat	no existing	often existing

Comparison: Speech vs Music Features

- ▶ Speech: bandwidth $\leq \sim 4$ kHz (telephone) to 7 kHz (wideband); low harmonicity in fricatives
- ▶ Music: wider bandwidth up to 20 kHz; strong harmonicity (fundamental + overtones)
- ▶ Speech: moderate ZCR in unvoiced consonants; music: lower average ZCR in sustained notes
- ▶ Speech: energy modulated by syllables ($\sim 3\text{--}5$ Hz rhythm); music: regular beat (60–180 BPM)
- ▶ Spectral centroid: speech typically 300–1500 Hz; music varies widely (200–4000 Hz)
- ▶ Classification strategy: bandwidth + harmonicity + silence ratio often separate speech from music

Audio Classification

▶ Step-by-step Classification

- ▶ At each step, one attribute of the sound is used separately to classify the sound
- ▶ Each attribute acts as a filter
- ▶ The order in which the attributes are selected for classification is very important, depending on the complexity as well as the ability to recognize the different sounds of each attribute
- ▶ Features with low complexity and high classification capability will be used first.
- ▶ The classification process can go through multiple steps with multiple attributes used, or it can use only one specific attribute.

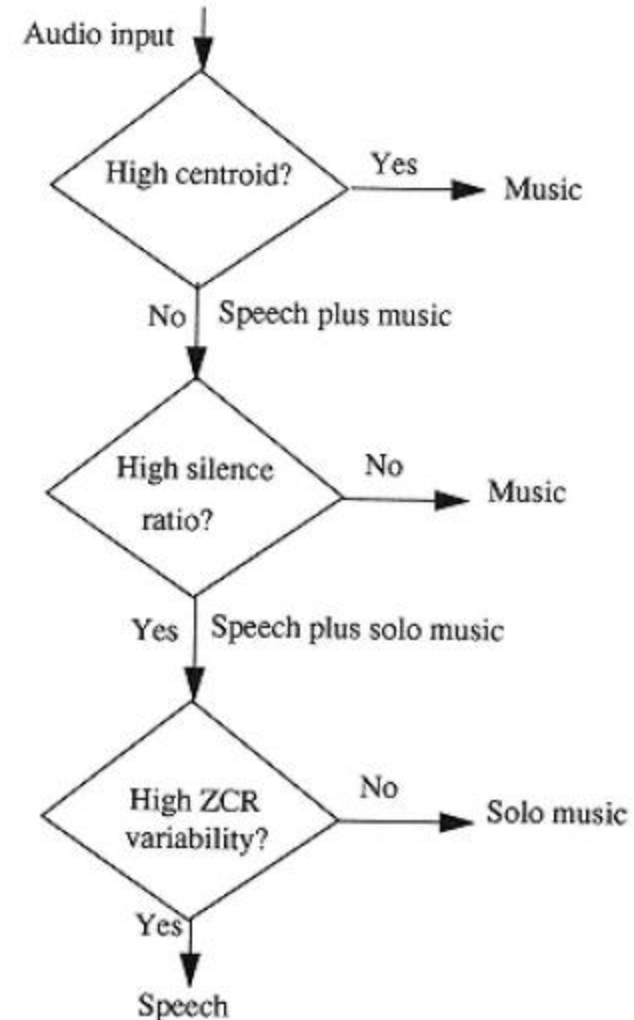
Step-Wise vs Attribute-Vector Classification

- ▶ Step-wise (decision tree): cheap features first — silence ratio → ZCR → bandwidth → harmonicity
- ▶ Order matters: place features with highest information gain and lowest cost early
- ▶ Lu & Hankinson (1998): example hierarchy filtering speech/music/noise/environmental sounds
- ▶ Attribute vector: concatenate all features → single N-dimensional vector per segment
- ▶ Scheirer & Slaney (1997): 13-D feature vector + Gaussian mixture model (GMM) classifier
- ▶ Vector approach: better accuracy, higher compute; enables k-NN and clustering in feature space



Step-by-step Audio Classification Example

► Lu và Hankinson 1998

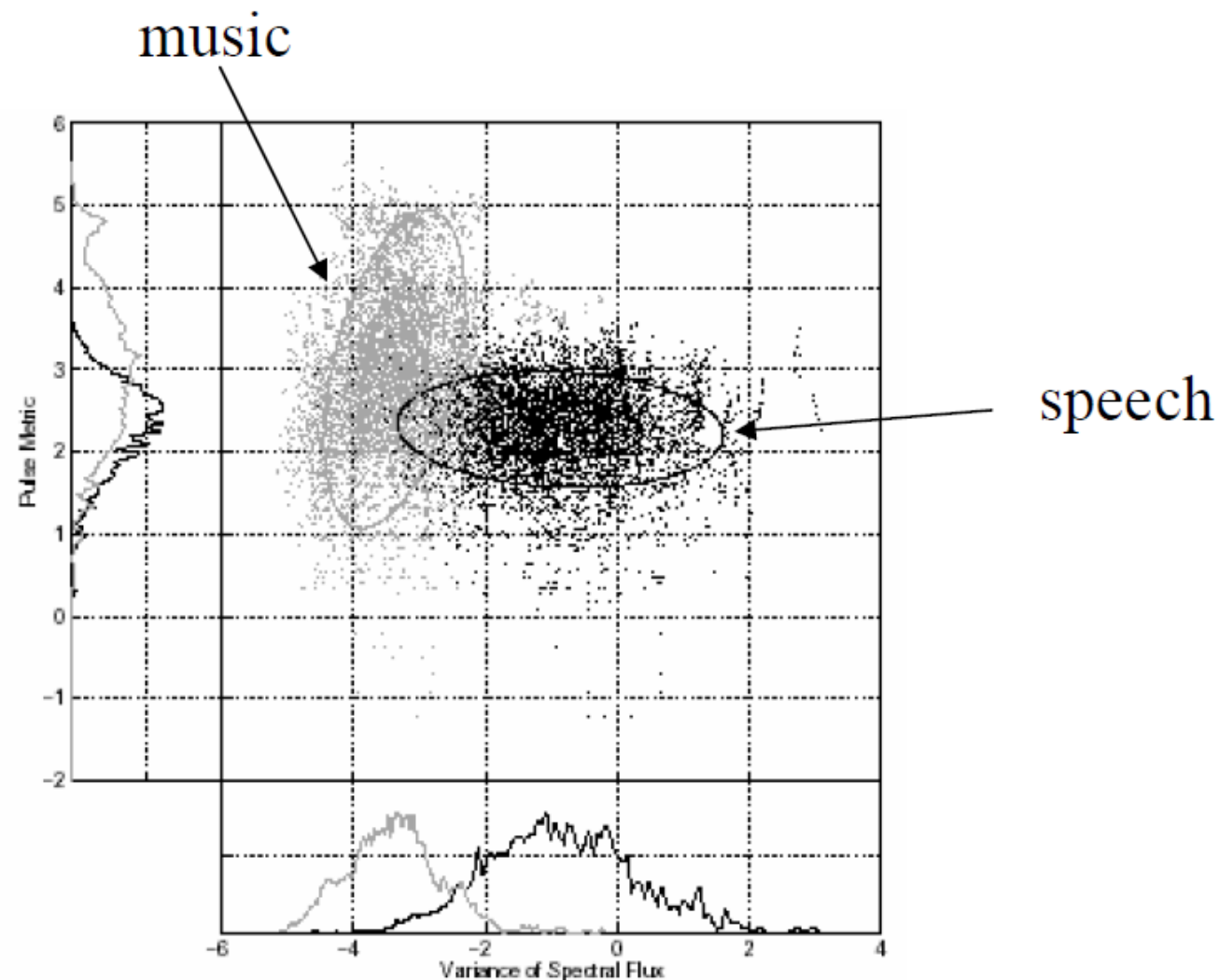


Audio classification

- ▶ **Attribute vector-based classification**
 - ▶ A set of attributes stitched together into a vector
 - ▶ Requires more computation, but delivers better results than using single attributes
 - ▶ Vector-space queries can be used to evaluate similarities between audio recordings
 - ▶ Sounds can also be clustered based on attribute vectors.

Audio classification based on attribute vectors

- Scheirer and Stanley 1997



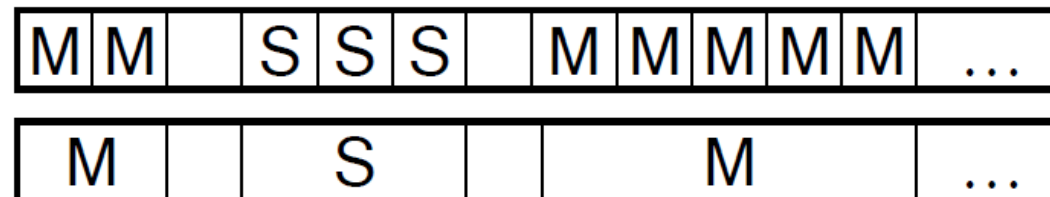
Examples of sound types

► Liu and Wan 2001

Class name	Number of files	Class name	Number of files
1.Speech	131	Mammals	174
Female	66	3.2Effect	344
Male	54	Alarm	17
Mixture	11	Beat	42
2.Music	458	Bells	50
2.1Instruments	365	Car	12
Brass	44	Clock	15
Keyboard	6	Explosion	44
Percussion	125	Gun	25
String	139	Machines	110
Synthesis	14	Telephone	29
Woodwind	37	3.3Nature	41
2.2Melody	36	Storm	11
Melody	36	Thunder	5
2.3Song	57	Water	20
Cartoon Song	7	Wind	5
Normal Song	50	3.4People	44
3.Sound	649	Applause	8
3.1Animal	220	Laughter	36
Birds	46	Total	1238

Audio Segmentation

- ▶ An audio file typically consists of multiple segments that contain different types of audio or different contents
- ▶ Audio files can be divided into separate segments of each type of audio
- ▶ Method:
 - ▶ Split the audio file into small segments, then use classification methods to classify the audio within those segments
 - ▶ Adjacent audio segments are merged together if they are of the same type



Audio Segmentation Algorithm

- ▶ Goal: partition audio into homogeneous segments (speech/music/noise/speaker change)
- ▶ Algorithm:
 - ▶ 1. Split into fixed frames (e.g., 20 ms)
 - ▶ 2. Extract feature vector per frame
 - ▶ 3. Classify each frame → label sequence
 - ▶ 4. Merge adjacent frames with same label → segments
 - ▶ 5. Apply minimum duration constraint (remove segments < 200 ms)
- ▶ Output: list of (start_time, end_time, class_label) stored as structural metadata



Audio Recognition and Retrieval

- ▶ Use speech recognition techniques to convert voice tones to text, and then use IR systems to store and query
 - ▶ Speech Recognition
 - ▶ Basics of Automatic Speech Recognition (ASR)
 - ▶ Advanced methods
 - ▶ Techniques based on the hidden Markov model (HMM)
 - ▶ Based on artificial neural networks
 - ▶ Speaker Recognition

Speech Retrieval: ASR + Information Retrieval

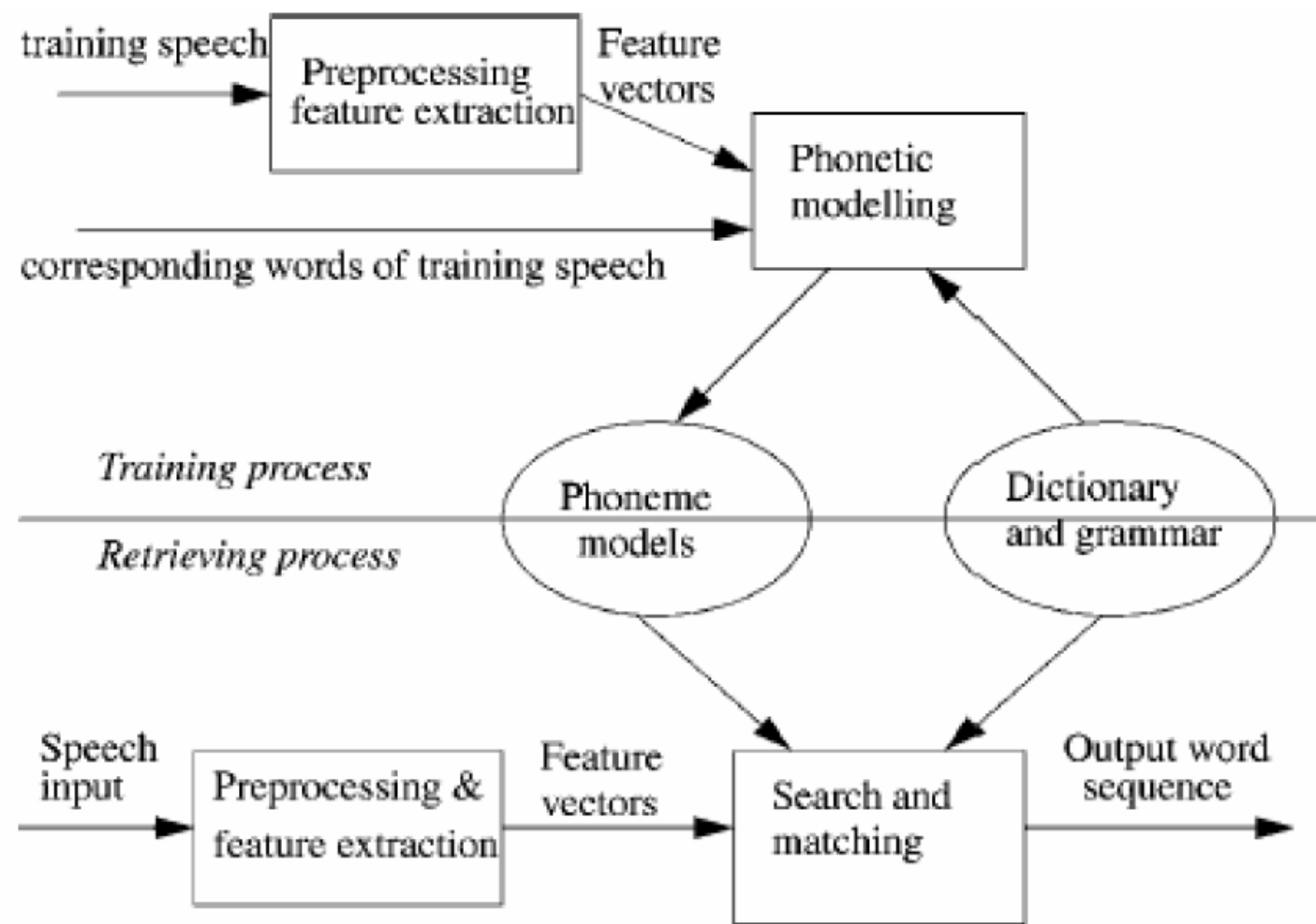
- ▶ Index pipeline: audio → ASR → transcript → tokenize → inverted index (terms → doc IDs + timecodes)
- ▶ Positional index: store (docID, start_time, end_time) per term for 'find when XYZ discussed'
- ▶ Speaker diarization: segment by speaker before ASR → metadata 'Speaker A said ... at 12:30'
- ▶ Limitations: domain vocabulary, noise, accents, homophones affect recall/precision
- ▶ Hybrid: combine ASR text index with audio feature index for robust retrieval
- ▶ Modern systems: Whisper (OpenAI) for ASR + Elasticsearch for full-text search over transcripts



Automatic Speech Recognition (ASR) Pipeline

- ▶ Input: audio waveform → preprocessing (noise reduction, normalization)
- ▶ Feature extraction: MFCC + delta + delta-delta per frame (feature vector)
- ▶ Acoustic model: HMM or neural network (CTC, Transformer) maps features → phonemes/characters
- ▶ Language model: n-gram or neural LM resolves ambiguous word sequences
- ▶ Decoder: finds most probable word sequence (Viterbi beam search or attention mechanism)
- ▶ Output: transcript text → indexed in inverted file for keyword Boolean/ranked search

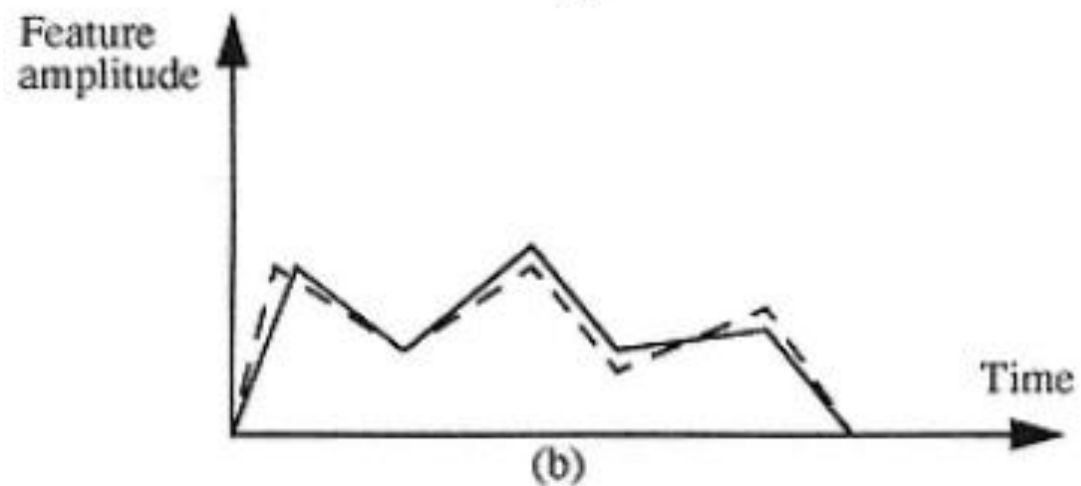
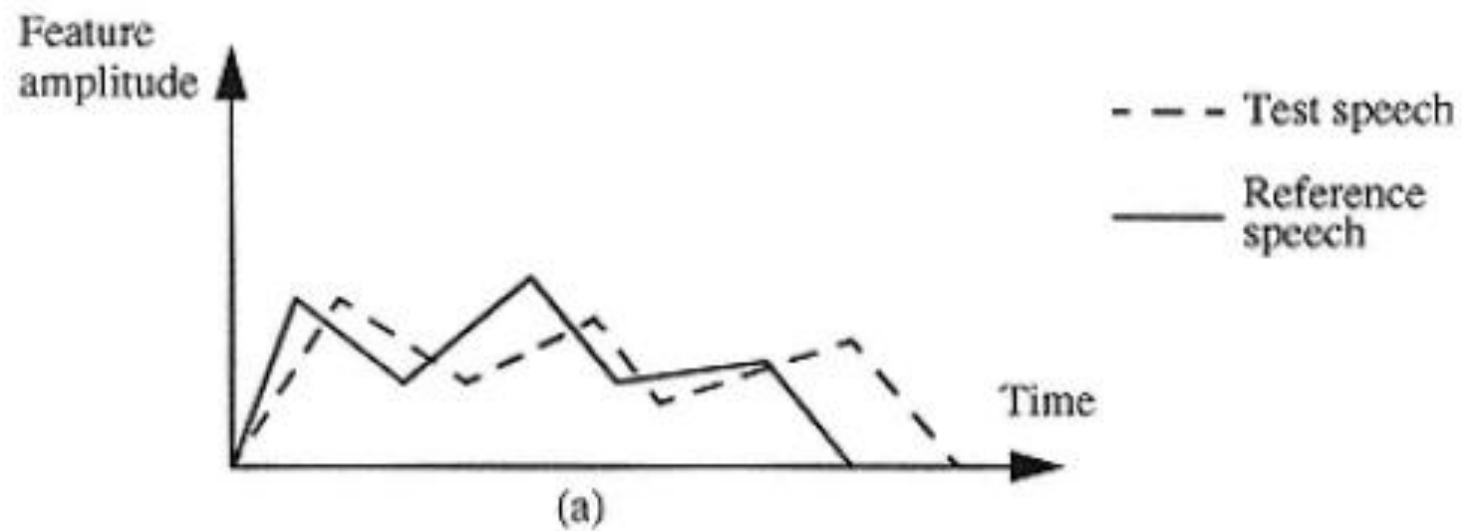
Automatic Speech Recognition



Limitations of ASR

- ▶ Depends on the topic
- ▶ Time-dependent
- ▶ Affected by interference
- ▶ The difference between the sounds of single words and the sounds of continuous spoken words in sentences
- ▶ The difference between pronunciation and spoken sounds
- ▶ Depends on vocabulary set size

Limitations of ASR



ASR Limitations — Summary and Mitigations

- ▶ Domain dependency: medical/legal jargon reduces accuracy → custom language model
- ▶ Noise/interference: overlap, background music → beamforming, source separation preprocessing
- ▶ Continuous vs isolated speech: coarticulation complicates word boundaries → context-aware LM
- ▶ Accent/pronunciation variation: diverse training data, speaker adaptation
- ▶ Large vocabulary: out-of-vocabulary (OOV) words missed → subword (BPE) tokenization
- ▶ Mitigation for retrieval: keep audio feature index as backup; confidence scores filter low-quality ASR

Music storage and retrieval

- ▶ There are two types of music data
 - ▶ Structured music and sound effects
 - ▶ Sampling-based music
 - ▶ Attribute vector-based queries
 - ▶ Retrieval based on pitch

Music Retrieval — Query Types

- ▶ Metadata query: search by title, artist, album (ID3 tags) — exact match, not content-based
- ▶ Query-by-example (QBE): submit audio clip → extract features → k-NN in feature database
- ▶ Query-by-humming (QbH): user hums melody → pitch contour extraction → DTW match against DB
- ▶ Query-by-synthetic (structured): MIDI note sequence → exact or approximate match in MIDI DB
- ▶ Audio fingerprinting (Shazam): constellation map of spectral peaks → robust hash lookup
- ▶ MMDB design: store multiple index layers — metadata + MFCC vector + fingerprint hash



Structured Music

- ▶ Can be represented by a set of statements or algorithms
- ▶ Example:
 - ▶ MIDI
 - ▶ MPEG-4
- ▶ Developed for audio transmission, sound reproduction, not for the purpose of querying audio data. However, the structure of the sounds is very convenient in retrieving audio data.
- ▶ Suitable for exact matched retrieval.

Sampling-based music

- ▶ Queried based on their attributes
 - ▶ Classify audio based on attributes
 - ▶ Retrieval by comparing attributes

Feature		Mean	Variance	Importance
Duration		2.71982	0.191312	6.21826
Loudness:	Mean	-45.0014	18.9212	10.3455
	Variance	200.109	1334.99	5.47681
	Autocorrelation	0.955071	7.71106e-05	108.762
Brightness:	Mean	6.16071	0.0204748	43.0547
	Variance	0.0288125	0.000113187	2.70821
	Autocorrelation	0.715438	0.0108014	6.88386
Bandwidth:	Mean	0.363269	0.000434929	17.4188
	Variance	0.00759914	3.57604e-05	1.27076
	Autocorrelation	0.664325	0.0122108	6.01186
Pitch:	Mean	4.48992	0.39131	7.17758
	Variance	0.207667	0.0443153	0.986485
	Autocorrelation	0.562178	0.00857394	6.07133

Class Model for Laughter (Wold et al., 1996)

Sampling-Based Music and Audio Fingerprinting

- ▶ MP3/WAV recordings: no explicit note structure — must extract features from samples
- ▶ Classification: genre, mood, instrument ID using attribute vectors + machine learning
- ▶ Pitch-based retrieval: extract f_0 contour → DTW alignment against database contours
- ▶ Fingerprinting: peaks in spectrogram → hash pairs (t_1, f_1, t_2, f_2) → inverted hash table lookup
- ▶ Robust to noise and compression — used in Shazam, SoundHound mobile apps
- ▶ Index structure: hash → $\{(\text{song_id}, \text{time_offset})\}$ for identification within seconds



Summary — Audio Indexing and Retrieval

- ▶ Audio IR requires classification (speech/music/noise) before choosing indexing strategy
- ▶ Time-domain and frequency-domain features enable content-based similarity search
- ▶ Segmentation produces structural metadata linking features to time intervals
- ▶ Speech: ASR → text inverted index; Music: MIDI structure or sampling-based fingerprints
- ▶ Modern systems combine metadata, feature vectors, and text indexes in layered architecture
- ▶ Next topics: image/video indexing, cross-media retrieval, deep learning audio embeddings

